Reviewer Pack — Minimal Rank of Cocomplexes and Circuit Monotone Width

Entry 895f842e21c0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 06:32:25 UTC

Minimal Rank of Cocomplexes and Circuit Monotone Width

Entry ID: `895f842e21c0` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 06:32:25 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Cocomplex Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every boolean circuit C with monotone gates and size n, the minimal rank of its associated cocomplex, denoted as mrank(C), is linearly correlated with its circuit monotone width w_mon(C), such that mrank(C) = Θ(w_mon(C)).

Rationale (proposer's reasoning):

> Cocomplex theory provides a geometric interpretation of boolean functions. The minimal rank of the cocomplex for a function could potentially reveal structural properties related to the complexity of computing the function, such as its monotone width. Exploring this connection might uncover new insights into circuit complexity.

Taxonomy category: `Geometric_Theoretic` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f7ca6d59b1c4a03f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the conjecture that minimal rank of cocomplexes is linearly correlated with monotone width, support is indicated by a Pearson correlation coefficient (r) greater than or equal to 0.8 for at least 24 out of 30 seeds, with the mean absolute difference between mrank(C) and w_mon(C) being less than or equal to 3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "cocomplex theory" AND "circuit monotone width" - "minimal rank of cocomplexes" AND "circuit size" - "0-linear relationship" AND ("cocomplex" OR "circuit monotone width")

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_boolean_circuit(n):
    if n < 1:
        return []

    circuit = []
    inputs = [0] * (n - 1)
    for i in range(1, n):
        gate_type = random.choice(['OR', 'AND'])
        if gate_type == 'OR':
            inputs.append(inputs[-1] | inputs[-2])
        else:
            inputs.append(inputs[-1] & inputs[-2])
        circuit.append((gate_type, inputs[-3], inputs[-2]))
    return circuit

def construct_cocomplex(circuit):
    cocomplex = []
    for gate in circuit:
        if gate[0] == 'OR':
            cocomplex.append([gate[1], gate[2]])
        else:
            cocomplex.append([gate[1], gate[2]])
    return cocomplex

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find the pivot
        max_row = i
```

```
            for j in range(i + 1, n):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

            # Eliminate below the pivot
            for j in range(i + 1, n):
                factor = Fraction(matrix[j][i], matrix[i][i])
                for k in range(n):
                    matrix[j][k] -= factor * matrix[i][k]

        # Back-substitute to find the solution
        solution = [0] * n
        for i in range(n - 1, -1, -1):
            solution[i] = Fraction(matrix[i][-1], matrix[i][i])
            for j in range(i):
                matrix[j][-1] -= matrix[j][i] * solution[i]
        return solution

    def minimal_rank(cocomplex):
        n = len(cocomplex)
        identity_matrix = [[Fraction(0, 1)] * n + [Fraction(1, 1) if i == j else Fraction(0, 1) for j
        augmented_matrix = [row + cocomplex[i] for i, row in enumerate(identity_matrix)]
        rank = gaussian_elimination(augmented_matrix)
        return sum(1 for x in rank if x != Fraction(0, 1))

    def circuit_monotone_width(circuit):
        width = 0
        for gate in circuit:
            inputs = [gate[1], gate[2]]
            width = max(width, len(inputs))
        return width

    def run_trial(seed: int) -> dict:
        random.seed(seed)

        n_max = 40
        instances_tested = 30
        metric_values = []
        conjecture_holds = True
        counterexample = ""

        for n in [5, 10, 15, 20, 30, 40]:
            for _ in range(instances_tested):
                circuit = generate_boolean_circuit(n)
                cocomplex = construct_cocomplex(circuit)
                mrank = minimal_rank(cocomplex)
                w_mon = circuit_monotone_width(circuit)

                metric_values.append((mrank, w_mon))

        if len(metric_values) < 180:
            return {
                "metric_name": "min_rank_vs_w_mon",
                "metric_value": None,
                "instances_tested": instances_tested,
```

```
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "insufficient_data"
        }

    mrank_values, w_mon_values = zip(*metric_values)
    correlation_coefficient = sum((m - mean_mrank) * (w - mean_w_mon) for m, w in zip(mrank_values
    mean_abs_diff = sum(abs(m - w) for m, w in zip(mrank_values, w_mon_values)) / len(metric_value

    if correlation_coefficient < 0.8 or mean_abs_diff > 3:
        conjecture_holds = False
        counterexample = "correlation_too_low_or_mean_abs_diff_too_high"

    return {
        "metric_name": "min_rank_vs_w_mon",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif sum(1 for r in results if not r["conjecture_holds"]) >= 8:
        print(f"RESULT: FALSIFIED counterexample=\"correlation_too_low_or_mean_abs_diff_too_high\"
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4bab8408.py", line 131, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4bab8408.py", line 93, in run_trial
```

```
    mrank = minimal_rank(cocomplex)
            ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4bab8408.py", line 70, in minimal_rank
    rank = gaussian_elimination(augmented_matrix)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4bab8408.py", line 54, in gaussian_elim
    factor = Fraction(matrix[j][i], matrix[i][i])
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means it did not complete its execution to provide a Pearson correlation coefficient and the mean absolute difference between mrank(C) and w_mon(C). Therefore, we cannot confirm or falsify the conjecture based on the provided results. | next: Re-run the test with proper error handling to ensure that it completes without crashing. If the test passes, re-evaluate the results against the pre-registered support and falsification conditions.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18442
2	preregistration	ollama_remote	glm4:latest	0	0	11194
3	novelty	ollama_remote	glm4:latest	0	0	10804
4	novelty	ollama_remote	glm4:latest	0	0	9095
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24203
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11096
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15215
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16652
9	judge	ollama_remote	glm4:latest	0	0	12320

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 129021 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/895f842e21c0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/895f842e21c0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/895f842e21c0.tar.gz` (if generated)